

APRENDIX AAA ## Plano mestre de produto e implementação ### Do primeiro contacto com programação ao nível muito avançado — local-first, privado e utilizável offline **Versão do plano:** 1.0 **Base técnica:** Appendix 0.18.0 **Data:** 2 de agosto de 2026 **Âmbito:** Windows, Android e preparação iOS

Sumário executivo

O objetivo é transformar o Appendix num produto integrado de aprendizagem que consiga orientar uma pessoa desde o zero até áreas avançadas de engenharia de software, dados e inteligência artificial, sem depender de contas, serviços cloud ou pesquisa constante na Internet.

O produto final não deve ser apenas uma coleção de funcionalidades. Deve funcionar como um sistema coordenado que:

1. sabe o que o utilizador já domina;
2. escolhe o melhor próximo passo e explica a escolha;
3. oferece imediatamente a teoria, exemplo, exercício, IDE ou revisão necessária;
4. mede aprendizagem real e retenção, não apenas cliques ou tempo aberto;
5. mantém cursos, dicionário, cards, bibliografia, pesquisa, tutor, projetos e jogos ligados ao mesmo grafo de competências;
6. funciona integralmente offline depois da instalação;
7. recebe, quando autorizado, atualizações de conteúdo auditáveis e reversíveis;
8. preserva privacidade, progresso e código exclusivamente no dispositivo.

A ordem recomendada começa pela experiência visual e pela instrumentação pedagógica; só depois expande cursos, pesquisa e conteúdos. O debugger e o tutor são construídos sobre contratos já estabilizados. A caixa multimodal de texto/imagem fica deliberadamente perto do fim: depende de OCR, pesquisa, tutor, IDE, segurança e experiência mobile já estarem maduros. Os jogos continuam isolados na aba **Games** e só recebem polimento depois de todas as funcionalidades pedagógicas serem validadas.

Este é um programa de produto, não uma alteração única. A execução deve ocorrer por incrementos verticais utilizáveis, com migrações reversíveis, feature flags e gates de qualidade. Nenhuma fase avança para produção apenas porque o código "existe". Tem de cumprir os critérios de aceitação, desempenho, acessibilidade, segurança, migração e funcionamento offline definidos neste documento.

Sumário executivo

1. Visão do produto

1.1 Promessa central

1.2 Públicos principais

1.3 Princípios não negociáveis

2. Estado de partida e lacunas

3. Estratégia de execução

3.1 Ordem crítica

3.2 Modelo de trabalho

3.3 Fluxos paralelos permitidos

3.4 Definition of Done comum

4. Arquitetura-alvo

4.1 Componentes

- 4.2 Separação de rede
- 4.3 Entidades novas ou revistas
- 5. Roadmap global
- 6. Fase 0 — Fundação, baseline e governança
 - Sprint 0.1 — Baseline reproduzível
 - Sprint 0.2 — Observabilidade local e feature flags
 - Critérios de aceitação
- 7. Fase 1 — Design system e experiência fluida
 - Sprint 1.1 — Design system Appendix
 - Sprint 1.2 — Navegação adaptativa
 - Sprint 1.3 — Persistência e acessibilidade
 - Gate UX-1
- 8. Fase 2 — Progresso e personalização
 - Sprint 2.1 — Evidência de aprendizagem
 - Sprint 2.2 — Mastery Engine v2
 - Sprint 2.3 — Planeador pessoal
 - Dashboard pretendido
 - Gate LEARN-1
- 9. Fase 3 — Academia Appendix
 - 9.1 Modelo curricular
 - 9.2 Percursos de fundação
 - 9.3 Percursos aplicados e avançados
 - 9.4 Estratégia de conteúdo
 - Metas de cobertura para o 1.0
 - Gate CURR-1
- 10. Fase 4 — Pesquisa inteligente e base de conhecimento
 - Sprint 4.1 — Pesquisa lexical estruturada
 - Sprint 4.2 — Pesquisa semântica
 - Sprint 4.3 — Compreensão e reranking
 - Sprint 4.4 — Benchmark e explicabilidade
 - Leitor v2
 - Gate SEARCH-1
- 11. Fase 5 — Conteúdo amplo e atualização semanal
 - 11.1 Registo de fontes confiáveis
 - 11.2 Formato .apxpack
 - 11.3 Pipeline semanal
 - 11.4 Experiência do utilizador
 - Gate UPDATE-1
- 12. Fase 6 — IDE, debugger e ferramentas de engenharia
 - 12.1 Editor

- 12.2 Debugger sandboxed
- 12.3 Testes e correção
- 12.4 Ferramentas pedagógicas
 - Gate IDE-1
- 13. Fase 7 — Dicionário, cards e bibliografia
 - 13.1 Dicionário v2
 - 13.2 Cards v2
 - 13.3 Bibliografia orientada
 - Gate CONTENT-UI-1
- 14. Fase 8 — Tutor offline
 - 14.1 Tutor fundamentado
 - 14.2 Estratégias pedagógicas
 - 14.3 Guardrails
 - Gate TUTOR-1
- 15. Fase 9 — Projetos e portfólio local
 - Gate PROJECT-1
- 16. Fase 10 — Desktop, Android e iOS
 - Desktop
 - Android
 - iOS
 - Transferência PC–mobile
 - Gate MOBILE-1
- 17. Fase 11 — Caixa multimodal de texto e imagem
 - 17.1 Entradas
 - 17.2 Pipeline de imagem
 - 17.3 Análise estrutural
 - 17.4 Ações apresentadas
 - 17.5 Experiência
 - 17.6 Segurança e privacidade
 - 17.7 Testes multimodais
 - Gate MULTI-1
- 18. Fase 12 — Games e pausas
 - Gate GAMES-1
- 19. Fase 13 — Hardening e release 1.0
 - Segurança
 - Desempenho
 - Qualidade
 - Release
 - Gate RELEASE-1
- 20. Matriz de testes

21. Indicadores de produto locais

22. Riscos e mitigação

23. Planeamento e estimativa

 Marcos de release sugeridos

24. Próximo passo imediato

25. Decisão final de estratégia

1. Visão do produto

1.1 Promessa central

“Abre o Appendix, escolhe um objetivo e recebe o melhor próximo passo — com explicação, prática, correção e progresso mensurável — sem precisar de procurar noutro lugar.”

1.2 Públicos principais

Perfil	Necessidade	Experiência proposta
Iniciante absoluto	Não sabe por onde começar	Diagnóstico suave, literacia digital e percurso guiado
Estudante	Consolidar matérias e preparar avaliações	Plano por data, testes, revisão e explicações alternativas
Reconversão profissional	Aprender competências empregáveis	Percurso por função, projetos e portefólio local
Programador intermédio	Eliminar lacunas e aprofundar	Diagnóstico curto, mapas de dependências e prática avançada
Especialista	Consulta e atualização	Pesquisa técnica, bibliografia orientada e packs temáticos
Utilizador offline	Aprender sem ligação estável	Conteúdo, IDE, tutor e progresso integralmente locais

1.3 Princípios não negociáveis

- **Local-first e anónimo:** sem login obrigatório, perfil e progresso locais.
- **Offline por defeito:** o núcleo não depende de serviços externos para ensinar ou corrigir.
- **Prática antes de excesso de teoria:** conteúdo curto, demonstração e aplicação imediata.
- **Segurança por fronteiras:** código do utilizador apenas no sandbox; OCR e conteúdo externo nunca são executados.
- **Explicabilidade:** recomendações, classificações e correções indicam o motivo.
- **Proveniência:** todo o conteúdo indica origem, versão, licença e data de revisão.
- **Conteúdo original:** fontes externas informam; não se copiam publicações protegidas.
- **Progressive disclosure:** o principiante vê apenas o necessário; opções avançadas aparecem quando são úteis.
- **Paridade por capacidade:** desktop e mobile podem ter layouts distintos, mas preservam o mesmo modelo pedagógico.

- **Qualidade antes de volume:** nenhuma meta numérica justifica conteúdo incorreto ou descontextualizado.

2. Estado de partida e lacunas

O release 0.18.0 já fornece uma base importante: SQLite cifrado, cursos iniciais, grafo A1, orquestração A2, pesquisa híbrida, leitor, cards autorais, dicionário, IDE sandboxed, progresso, Pomodoro, ingestão, desktop Windows, Android e Games.

As principais lacunas para chegar ao nível pretendido são:

- navegação e design ainda não formam um sistema visual completo;
- o progresso necessita de calibração longitudinal e recomendações explicáveis;
- o catálogo de cursos ainda não cobre um percurso completo do zero ao avançado;
- a pesquisa precisa de benchmark, compreensão de intenção e personalização;
- a base precisa de versionamento de conhecimento, conteúdo e tecnologias;
- a atualização semanal ainda não existe como pipeline assinado e reversível;
- o IDE precisa de debugger, testes, profiler e visualizadores pedagógicos;
- tutor, bibliografia, cards e dicionário precisam de agir como superfícies do mesmo conhecimento;
- falta um sistema de projetos e portefólio;
- mobile precisa de ergonomia própria para escrita e leitura prolongadas;
- falta a entrada multimodal de texto/imagem para explicar ou ajudar a construir algoritmos;
- a qualidade deve passar de testes funcionais pontuais para gates contínuos de produto.

3. Estratégia de execução

3.1 Ordem crítica

A sequência correta é:

1. congelar contratos e criar métricas;
2. construir design system, navegação e acessibilidade;
3. melhorar o modelo de aprendizagem e o planeador;
4. formalizar cursos e o grafo curricular;
5. melhorar pesquisa, dados e atualização de conteúdos;
6. elevar o IDE com linguagem inteligente e debugger;
7. unificar cards, dicionário e bibliografia;
8. construir o tutor offline;
9. adicionar projetos e portefólio;
10. fechar paridade e ergonomia mobile/iOS;

11. adicionar a caixa multimodal de texto/imagem;
12. polir Games;
13. executar hardening e preparar o release 1.0.

Esta ordem evita construir interfaces sobre contratos instáveis e impede que a funcionalidade multimodal se torne um “chat isolado” sem ligação ao percurso pedagógico.

3.2 Modelo de trabalho

- Sprints curtos, cada um com uma fatia utilizável ponta a ponta.
- Feature flags locais para funcionalidades incompletas.
- Migrações SQLite incrementais, testadas sobre cópias de bases antigas.
- Conteúdo separado do código através de packs versionados.
- Benchmarks fixos para pesquisa, OCR, recomendação e desempenho.
- Demonstração funcional no fim de cada sprint em Windows e Android quando aplicável.
- iOS validado em macOS/Xcode nos gates mobile; nunca se assume que um APK serve para iOS.
- O progresso real do utilizador nunca é eliminado durante atualização ou migração.

3.3 Fluxos paralelos permitidos

Depois da fundação, três fluxos podem avançar em paralelo:

Fluxo	Responsabilidade	Dependência principal
Produto/UI	Design system, navegação, acessibilidade, desktop/mobile	Contratos de apresentação
Aprendizagem/conteúdo	Grafo, cursos, exercícios, tutor, atualização	Modelo de domínio
Plataforma	Pesquisa, BD, IDE, sandbox, builds, desempenho	Migrações e interfaces estáveis

Os fluxos voltam a convergir nos gates de integração. Não se devem criar três implementações diferentes de progresso, pesquisa ou conteúdos.

3.4 Definition of Done comum

Uma funcionalidade só fica concluída quando:

- tem contratos tipados e migração quando necessária;
- funciona sem Internet;
- tem testes unitários, integração e caminho funcional;
- trata vazio, erro, indisponibilidade e cancelamento;
- preserva dados existentes;
- funciona por teclado no desktop e por toque no mobile;
- respeita tema claro/escuro e escala de letra;

- não bloqueia a UI durante tarefas demoradas;
- apresenta linguagem clara e proveniência;
- não permite que conteúdo ou código não confiável atravesse o sandbox;
- possui métricas locais suficientes para diagnosticar falhas sem recolher telemetria externa;
- atualiza documentação, instalador e self-test.

4. Arquitetura-alvo

4.1 Componentes

Componente	Função
Shell Desktop/Mobile	Navegação, estado visual, acessibilidade e interação
Learning Planner	Decide o melhor próximo passo e constrói o plano semanal
Mastery Engine	Estima domínio, retenção, velocidade, autonomia e confiança
Curriculum Graph	Cursos, unidades, conceitos, pré-requisitos e milestones
Exercise Engine	Exercícios fixos, variações e avaliações
IDE Service	Edição, análise, execução, debugging e testes
Tutor Service	Explicação local baseada em fontes aprovadas
Search Service	BM25F, vetores, grafo, reranking e personalização
Knowledge Store	Conteúdo, relações, fontes, versões, media e índices
Content Pipeline	Ingestão, deduplicação, licenças, qualidade e quarentena
Update Broker	Descoberta e importação semanal de packs assinados
Multimodal Assistant	Texto, imagem, OCR, estrutura e ajuda algorítmica
Secure Sandbox	Execução e debug isolados, sem rede
Local Vault	Chaves, progresso, projetos, backups e preferências

4.2 Separação de rede

Para preservar a promessa offline, o processo principal não deve navegar livremente na Internet.

Recomendação:

- **Core Appendix:** sem dependência de rede para qualquer função pedagógica.
- **Update Broker separado:** único componente autorizado a consultar fontes allowlisted.
- **Windows:** helper separado, executado apenas por agenda ou ação do utilizador.
- **Android:** duas opções suportadas: importar `.apxpack` transferido pelo PC ou build opcional com updater de rede.

- **iOS:** importação de pack pelo Files/Share Sheet; updaters conectados apenas numa build assinada que declare a capacidade necessária.

O Update Broker descarrega dados, nunca executáveis. Todos os packs são validados antes de o core os conseguir ler.

4.3 Entidades novas ou revistas

- `Concept`, `ConceptAlias`, `ConceptRelation`, `PrerequisiteEdge`;
- `Course`, `CourseVersion`, `Module`, `Lesson`, `LearningObjective`;
- `ExerciseTemplate`, `ExerciseVariant`, `TestCase`, `Rubric`;
- `LearningEvent`, `SkillEvidence`, `MasteryState`, `RetentionSchedule`;
- `StudyGoal`, `StudyPlan`, `PlanAdjustment`, `Intervention`;
- `Source`, `SourceSnapshot`, `LicenseRecord`, `Citation`;
- `ContentPack`, `PackManifest`, `PackDependency`, `PackRollback`;
- `SearchDocument`, `SearchFeature`, `SearchJudgment`;
- `CodeDiagnostic`, `DebugSession`, `Project`, `ProjectMilestone`;
- `OCRArtifact`, `SnippetAnalysis`, `AlgorithmDraft`.

Todos os registos de conhecimento devem possuir versão, proveniência, idioma, dificuldade, estado de qualidade e datas de criação/revisão.

5. Roadmap global

Fase	Resultado	Gate de saída
0. Fundação	Baseline, contratos, métricas e migrações seguras	Release atual reproduzível
1. Experiência	Design system e navegação fluida	Jornadas críticas sem bloqueios
2. Personalização	Modelo de domínio e plano semanal	Recomendações corretas e explicáveis
3. Academia	Cursos completos e grafo curricular	Percurso zero→avançado navegável
4. Pesquisa/BD	Pesquisa v2 e conhecimento versionado	Benchmark de relevância aprovado
5. Atualizações	Packs semanais assinados	Instalação, quarentena e rollback aprovados
6. IDE	Debugger e ferramentas pedagógicas	Ciclo escrever→testar→corrigir completo
7. Superfícies	Cards, dicionário e bibliografia unificados	Conteúdo certo no contexto certo
8. Tutor	Tutor local, fundamentado e socrático	Respostas fiáveis e sem “invenção” silenciosa
9. Projetos	Projetos guiados e portefólio	Competências demonstradas em entregáveis

Fase	Resultado	Gate de saída
10. Mobile	Ergonomia, paridade e transferência segura	Jornadas completas em dispositivos reais
11. Multimodal	Texto/imagem para explicar e criar algoritmos	OCR confirmado e ajuda integrada no IDE
12. Games	Pausas acessíveis e controladas	Jogos isolados da pontuação pedagógica
13. Hardening	Segurança, performance e release 1.0	Todos os gates finais aprovados

6. Fase 0 — Fundação, baseline e governança

Sprint 0.1 — Baseline reproduzível

- Congelar o release 0.18.0 como fixture de migração.
- Mapear todas as jornadas atuais e respetivos pontos de falha.
- Criar testes de contrato para controller, repositories e serviços.
- Criar conjuntos de dados pequenos, médios e grandes.
- Registrar tempos de arranque, pesquisa, navegação, OCR e execução.
- Criar matriz Windows 10/11, escalas DPI e Android API/ABI.
- Tornar o build determinístico e gerar SBOM das dependências.

Sprint 0.2 — Observabilidade local e feature flags

- Event log local com códigos estáveis, sem dados sensíveis.
- Diagnóstico exportável e explicitamente autorizado.
- Feature flags persistidas e reversíveis.
- Relatório de crash local com limpeza de código e conteúdo privado.
- Health checks para BD, índices, packs, sandbox e chaves.

Critérios de aceitação

- Uma base 0.18.0 migra sem perder progresso ou projetos.
- O release pode ser construído de raiz por um único script.
- Cada serviço principal expõe health status.
- Nenhum log contém código, respostas ou texto privado por defeito.

7. Fase 1 — Design system e experiência fluida

Sprint 1.1 — Design system Appendix

- Tokens de cor, tipografia, espaçamento, elevação, radius e estados.
- Componentes partilhados: botão, campo, card, modal, tooltip, toast, tabela e skeleton.
- Temas escuro, claro, alto contraste e redução de movimento.
- Escala tipográfica para leitura longa, matemática e código.
- Ícones vetoriais consistentes e independentes de resolução.

Sprint 1.2 — Navegação adaptativa

- Sidebar recolhível no desktop.
- Bottom navigation no mobile.
- Breadcrumbs, voltar/avançar e histórico por sessão.
- **Ctrl+K** /command palette global.
- Deep links internos para curso, conceito, exercício, fonte e projeto.
- Estado de navegação único para impedir janelas duplicadas.

Sprint 1.3 — Persistência e acessibilidade

- Guardar posição de scroll, filtros, separadores e tamanho de painéis.
- Navegação completa por teclado.
- Touch targets mínimos e gestos configuráveis.
- Leitura por screen reader onde o toolkit permitir.
- Testes de contraste e foco.
- Estados vazios com próximo passo útil.

Gate UX-1

- Dashboard, curso, IDE, pesquisa, cards, dicionário e Games são alcançados em até duas ações.
- Nenhuma tarefa pesada bloqueia a UI por mais de 100 ms sem feedback.
- Layout aprovado em 100%, 125%, 150%, 175% e 200% DPI.
- Jornadas principais funcionam por teclado e toque.

8. Fase 2 — Progresso e personalização

Sprint 2.1 — Evidência de aprendizagem

Cada interação relevante gera evidência local normalizada:

- resposta correta/incorreta;
- tempo ativo e tempo total;
- número e tipo de dicas;
- tentativas;
- erro conceitual, sintático ou de distração;
- transferência para um contexto diferente;
- retenção em revisão posterior;
- qualidade do projeto.

Não se usa tempo aberto como sinónimo de aprendizagem.

Sprint 2.2 — Mastery Engine v2

- Bayesian Knowledge Tracing por conceito.
- IRT para dificuldade e discriminação de itens.
- Repetição espaçada dependente da recordação observada.
- Intervalos de confiança, não apenas percentagens absolutas.
- Detecção de pré-requisito em falta.
- Cold start com diagnóstico adaptativo.
- Fallback determinístico quando os dados ainda são insuficientes.

Sprint 2.3 — Planeador pessoal

- Objetivo, data, horas por semana e percentagem de avaliação.
- Plano semanal com blocos de aprender, praticar, rever e avaliar.
- Replaneamento automático perante atrasos ou progresso acelerado.
- Botão permanente **Melhor próximo passo**.
- Explicação curta da recomendação.
- Modo "Tenho 10/25/50/90 minutos".

Dashboard pretendido

- progresso global e por percurso;
- competências novas, consolidadas e em risco;
- retenção prevista;
- velocidade e consistência;
- horas efetivas e planeadas;
- próximos milestones;
- comparação apenas com o próprio histórico;
- resumo semanal em linguagem clara.

Gate LEARN-1

- Benchmark sintético confirma atualização monotónica e estável do domínio.
- Recomendações nunca saltam pré-requisitos bloqueantes.
- Cada recomendação possui reason code legível.
- O utilizador pode alterar ou ignorar o plano sem penalização.

9. Fase 3 — Academia Appendix

9.1 Modelo curricular

Estrutura fixa:

Percurso → Curso → Módulo → Unidade → Objetivo → Conceito → Evidência

Cada unidade inclui:

- pré-requisitos;
- objetivos observáveis;
- explicação essencial;
- exemplo executável;
- prática guiada;
- prática independente;
- revisão;
- teste de etapa;
- bibliografia essencial e avançada.

9.2 Percursos de fundação

1. Literacia computacional e resolução de problemas.
2. Lógica e pseudocódigo.
3. Python base.
4. Programação orientada a objetos.
5. Algoritmos e estruturas de dados.
6. Matemática para programação.
7. Testes, debugging e qualidade.
8. Git e organização de projetos.

9.3 Percursos aplicados e avançados

- Python avançado e performance;
- SQL, modelação e bases de dados;
- desenvolvimento Web e APIs;
- Django e FastAPI;
- HTML, CSS e JavaScript;
- Data Science e visualização;
- Machine Learning clássico;

- Deep Learning e redes neuronais;
- NLP;
- visão computacional;
- sistemas de recomendação;
- agentes autónomos;
- arquitetura e sistemas distribuídos;
- segurança aplicada;
- Go e React como packs posteriores, depois de sandboxes próprios.

9.4 Estratégia de conteúdo

- Consolidar primeiro Python, algoritmia, matemática e engenharia de software.
- Abrir SQL e dados quando o percurso-base estiver completo.
- Adicionar tecnologias através de packs independentes.
- Não marcar um curso como completo sem exercícios, avaliações e projeto.
- Usar templates para gerar variações, mas validar invariantes e soluções.
- Introduzir revisão editorial e amostragem automática.

Metas de cobertura para o 1.0

Metas de direção, subordinadas à qualidade:

- 12 percursos principais utilizáveis;
- 150–250 unidades ligadas por pré-requisitos;
- 2 000+ conceitos normalizados;
- 3 000+ exercícios aprovados, além de variações determinísticas;
- 1 000+ cards autorais;
- 5 000+ entradas/alias no dicionário;
- 1 000+ referências curadas com orientação por secção.

Gate CURR-1

- Um iniciante consegue concluir um percurso sem pesquisa externa.
- Não existem unidades órfãs ou ciclos inválidos no grafo.
- Todo o teste mede objetivos declarados.
- Conteúdo e solução passam validação técnica e pedagógica.

10. Fase 4 — Pesquisa inteligente e base de conhecimento

Sprint 4.1 — Pesquisa lexical estruturada

- SQLite FTS5/BM25F.
- Pesos separados para título, definição, código, fórmula, aliases e corpo.
- Tokenização técnica para `__init__`, `%`, `JOIN`, `async/await` e erros.
- Correção ortográfica sem destruir identificadores.
- Filtros por área, nível, tecnologia, tipo, data, fonte e versão.

Sprint 4.2 — Pesquisa semântica

- Embeddings quantizados e versionados.
- Índice ANN/HNSW persistente ou implementação equivalente compatível com o empacotamento.
- Pesquisa incremental após novos packs.
- RRF para combinar lexical, semântica e grafo.
- MMR para diversidade.

Sprint 4.3 — Compreensão e reranking

- Classificador de intenção: definição, comparação, tutorial, erro, exemplo, bibliografia ou exercício.
- Expansão por aliases, acrônimos e relações.
- Cross-encoder leve para top-N.
- Personalização por nível, percurso e domínio sem ocultar resultados relevantes.
- Confidence score calibrado.

Sprint 4.4 — Benchmark e explicabilidade

- Conjunto de consultas reais e julgamentos de relevância.
- Casos de termos simples, símbolos, código e mensagens de erro.
- Métricas Recall@K, MRR e nDCG@10.
- “Porque apareceu” em cada resultado.
- Zero conteúdo em quarentena nos resultados.

Leitor v2

- Preservação de títulos, listas, código, tabelas, imagens e fórmulas.
- Resumo da fonte aberta, sem recitar referências no corpo.
- Modo simplificado centrado em matemática e pré-requisitos.

- Tutor apenas por duplo clique/toque ou seleção explícita.
- Comparação lado a lado entre fontes.
- Notas e bookmarks locais.

Gate SEARCH-1

- nDCG@10 alvo $\geq 0,80$ no benchmark aprovado.
- P95 de pesquisa local inferior a 500 ms no dataset de referência.
- Cancelamento de pesquisa não deixa threads ou transações pendentes.
- Resultados apresentam fonte e nível de confiança.

11. Fase 5 — Conteúdo amplo e atualização semanal

11.1 Registo de fontes confiáveis

Cada adapter de fonte define:

- domínio allowlisted;
- tipo de conteúdo permitido;
- licença e política de reutilização;
- robots/rate limits;
- parser e versão;
- método de deteção de alterações;
- autoridade e nível de revisão.

Prioridade: documentação oficial, standards, PEPs, documentação de bibliotecas, DOI/Crossref, revistas reconhecidas e feeds oficiais. Preprints são identificados como tal e não recebem o mesmo peso de material revisto.

11.2 Formato `.apxpack`

Um pack contém:

- manifesto JSON canónico;
- versão e dependências;
- hashes SHA-256;
- assinatura Ed25519;
- conteúdo normalizado;
- media permitida;
- índice delta;
- licença/proveniência;

- script declarativo de migração de conteúdo, nunca código executável.

11.3 Pipeline semanal

1. Verificar manifesto remoto ou fontes allowlisted.
2. Descarregar apenas conteúdos alterados.
3. Validar tamanho, MIME, assinatura e hashes.
4. Impedir path traversal, zip bombs e conteúdo executável.
5. Extrair texto e metadata em staging.
6. Deduplicar por hash e semântica.
7. Classificar área, nível, versão e confiança.
8. Detetar fronteiras de publicações e contaminação.
9. Colocar tudo em quarentena.
10. Executar validações e amostragem editorial.
11. Promover transacionalmente.
12. Reindexar em background.
13. Permitir rollback para o pack anterior.

11.4 Experiência do utilizador

- Atualização semanal opt-in.
- Indicação de tamanho e fontes antes de descarregar.
- Opção “só quando ligado à corrente/Wi-Fi”.
- Feed “Novidades desta semana”.
- Alterações associadas aos cursos relevantes.
- Sem notificações excessivas.

Gate UPDATE-1

- Pack adulterado é sempre rejeitado.
- Instalação interrompida mantém a versão anterior operacional.
- Rollback restaura conteúdo e índices.
- Nenhum pack consegue introduzir Python, binários ou comandos executáveis.
- A aplicação continua integralmente utilizável sem atualizar.

12. Fase 6 — IDE, debugger e ferramentas de engenharia

12.1 Editor

- Realce sintático e números de linha.
- Indent guides e matching de delimitadores.
- Múltiplos ficheiros em projetos.
- Pesquisa/substituição.
- Formatter e organização de imports.
- Autocomplete sensível ao contexto.
- Hover/F1 com dicionário e documentação local.
- Diagnósticos incrementais com quick fixes.

12.2 Debugger sandboxed

- Breakpoints normais e condicionais.
- Step into, over, out e continue.
- Call stack.
- Variáveis locais e globais permitidas.
- Watches com avaliação AST restrita.
- Consola de debug controlada.
- Limites de passos, CPU, memória e output.
- Canal IPC autenticado entre UI e helper.
- Sem rede e sem acesso livre ao sistema de ficheiros.

O debugger deve usar tracing no processo isolado. Nunca se liga um debugger de rede genérico diretamente ao processo principal.

12.3 Testes e correção

- Runner de testes unitários.
- Testes ocultos e públicos separados.
- Diferença entre esperado e obtido.
- Cobertura por linha.
- Property-based tests para tópicos adequados.
- Rubricas para estilo, estrutura e resultado.
- Dicas em escada: localização → conceito → estratégia → exemplo parcial.

12.4 Ferramentas pedagógicas

- Visualizador de execução linha a linha.
- Visualizador de listas, pilhas, filas, árvores e grafos.
- Matrizes e tensores com shape/dtype.
- Profiler básico de CPU e memória.
- Explicação de complexidade temporal e espacial.
- Comparação entre duas soluções.
- Histórico local e recuperação de versões.

Gate IDE-1

- Crash, timeout e ciclo infinito não derrubam a aplicação.
- Breakpoints e inspeção funcionam em exercícios e projetos.
- O processo do utilizador não tem rede.
- Testes de escape do sandbox permanecem bloqueados.
- Rascunho e estado de debug sobrevivem ao fecho inesperado.

13. Fase 7 — Dicionário, cards e bibliografia

13.1 Dicionário v2

Cada entrada pode conter:

- definição curta e explicação extensa;
- sintaxe/assinatura;
- parâmetros e retorno;
- exemplo mínimo executável;
- erros frequentes;
- aliases e tradução;
- conceitos relacionados;
- nível e pré-requisitos;
- versões tecnológicas;
- fontes oficiais;
- botão **Experimentar no IDE**.

Atalhos: duplo clique, seleção, **F1**, command palette e pesquisa por mensagem de erro.

13.2 Cards v2

- Deck diário calculado pelo Mastery Engine.
- Curiosidade, revisão, output prediction, bugs e matemática.

- Interleaving entre tópicos próximos.
- “Já sabia”, “Útil”, “Confuso” e “Rever”.
- Explicação de por que o card apareceu.
- Código executável quando seguro.
- Conteúdo apenas de fontes aprovadas ou autoria Appendix.

13.3 Bibliografia orientada

Categorias:

- essencial agora;
- consulta rápida;
- aprofundamento;
- referência avançada;
- histórico/desatualizado.

Cada item indica porquê, secções relevantes, dificuldade, tempo estimado, conceitos cobertos e versão. O leitor deve permitir notas e checkpoints.

Gate CONTENT-UI-1

- Uma definição simples é encontrada offline em até duas ações.
- Nenhum card cruza áreas apenas por aparecer no mesmo PDF.
- Bibliografia nunca recomenda 700 páginas quando bastam duas secções.
- Feedback de cards altera a revisão sem manipular o utilizador.

14. Fase 8 — Tutor offline

14.1 Tutor fundamentado

- Recuperação exclusivamente sobre conteúdo aprovado.
- Resposta com evidências internas e fontes consultáveis.
- Nível de confiança e recusa quando não existe base suficiente.
- Cache cifrada de respostas derivadas.
- Modelo pequeno quantizado por defeito; packs opcionais para hardware capaz.

14.2 Estratégias pedagógicas

- Explicar de outra forma.
- Fazer pergunta socrática.
- Dar exemplo novo.
- Mostrar pré-requisito em falta.

- Simplificar matemática sem perder rigor.
- Analisar erro sem fornecer logo a solução.
- Criar mini-exercício de confirmação.

14.3 Guardrails

- Conteúdo recuperado é dado, não instrução executável.
- Respostas não alteram ficheiros sem ação explícita.
- O tutor não marca domínio apenas porque mostrou uma explicação.
- A solução integral de uma avaliação bloqueada não é revelada.
- O utilizador consegue inspecionar e apagar histórico.

Gate TUTOR-1

- Suite factual mede groundedness e cobertura.
- Perguntas fora da base produzem resposta honesta.
- O tutor adapta vocabulário sem alterar o significado técnico.
- Latência e consumo cabem no orçamento do dispositivo de referência.

15. Fase 9 — Projetos e portefólio local

- Projetos guiados por nível e área.
- Requisitos, milestones, testes e rubrica.
- Revisão de arquitetura, testes, qualidade e documentação.
- Histórico de versões local.
- Relatório das competências demonstradas.
- Exportação ZIP/Git apenas por ação explícita.
- Projetos capstone que cruzam várias unidades.
- Modo "briefing profissional" com requisitos incompletos controlados.

Gate PROJECT-1

- Pelo menos um projeto final por percurso principal.
- Avaliação reproduzível e explicável.
- Projeto exportado não contém chaves nem dados privados.
- O portefólio distingue exercícios guiados de trabalho autónomo.

16. Fase 10 — Desktop, Android e iOS

Desktop

- Painéis redimensionáveis e restaurados.
- Multi-monitor e DPI misto.
- Atalhos completos.
- Integração segura com ficheiros locais.
- Atualizador separado e manifesto de instalação.

Android

- IDE em landscape.
- Barra de símbolos `() [] {} : _ =` e indentação.
- Gestos configuráveis sem substituir botões acessíveis.
- Background work sujeito a bateria/rede.
- Notificações locais de revisão.
- Packs e backups pelo Storage Access Framework.

iOS

- Shell Toga/Briefcase validado num Mac real.
- File picker, Share Sheet e importação `.apxpack`.
- Assinatura e provisioning documentados.
- Testes em dispositivo; não assumir equivalência do simulador.

Transferência PC–mobile

- Exportação cifrada com versão e hash.
- Merge determinístico de progresso.
- Preview de conflitos.
- Transferência por ficheiro, QR para metadata pequena ou rede local explícita.
- Sem conta cloud obrigatória.

Gate MOBILE-1

- Curso, pesquisa, cards, dicionário, progresso e Games funcionam em dispositivo Android real.
- Import/export preserva dados e resolve duplicados.
- Build iOS é produzido e assinado apenas em macOS/Xcode.
- Consumo de bateria e armazenamento é medido.

17. Fase 11 — Caixa multimodal de texto e imagem

Esta fase é uma das últimas porque reutiliza OCR, dicionário, pesquisa, tutor, IDE, grafo e sandbox. A funcionalidade aparece como **Analisar trecho** e também pode ser aberta a partir do IDE.

17.1 Entradas

- texto escrito ou colado;
- código e pseudocódigo;
- imagem por upload no PC;
- imagem pela galeria/câmara no mobile;
- screenshot colado no desktop;
- recorte e rotação antes de processar;
- formatos PNG, JPEG e WebP com limites explícitos.

Nada é enviado para serviços externos.

17.2 Pipeline de imagem

1. Validar MIME, tamanho e dimensões reais.
2. Remover metadata EXIF desnecessária.
3. Corrigir orientação, perspectiva, contraste e ruído.
4. Detetar blocos, linhas, indentação e símbolos.
5. Executar OCR local quantizado.
6. Produzir texto com confiança por região.
7. Mostrar o resultado ao utilizador para confirmação/correção.
8. Só depois analisar o conteúdo.

O OCR nunca deve ocultar baixa confiança. Caracteres críticos como `0/0`, `1/l`, `:` e indentação são destacados quando ambíguos.

17.3 Análise estrutural

- Detecção de linguagem ou pseudocódigo.
- Normalização sem destruir o original.
- AST quando a linguagem é reconhecida.
- Control Flow Graph para funções e algoritmos.
- Inferência de inputs, outputs e invariantes.
- Detecção de estruturas, ciclos, condições e recursão.

- Estimativa explicada de complexidade.
- Pesquisa de conceitos e padrões relacionados.
- Detecção de erros provável com níveis de confiança.

17.4 Ações apresentadas

- **Explicar:** resumo, linha a linha e conceitos.
- **Ajudar a completar:** perguntas sobre objetivo e proposta incremental.
- **Transformar em algoritmo:** inputs, outputs, passos e pseudocódigo limpo.
- **Converter para Python:** rascunho enviado ao IDE, nunca executado automaticamente.
- **Encontrar problemas:** lógica, indentação, casos extremos e complexidade.
- **Criar testes:** exemplos normais, limites e casos inválidos.
- **Visualizar:** fluxo, variáveis e estruturas.
- **Ligar ao curso:** pré-requisitos e unidades relevantes.

17.5 Experiência

- Caixa única com separadores Texto/Imagem.
- Drag-and-drop no desktop e file picker/câmera no mobile.
- Preview lado a lado: original, OCR e explicação.
- Botão "Corrigir texto extraído".
- Perguntas de clarificação apenas quando alteram materialmente o resultado.
- Botão "Abrir no IDE" e "Guardar como nota/projeto".
- Histórico local apagável.

17.6 Segurança e privacidade

- Processamento local.
- Imagem temporária eliminada após análise salvo consentimento explícito.
- Limites de pixels, memória e tempo.
- Decoders isolados quando possível.
- Texto extraído tratado como dado não confiável.
- Nenhuma execução automática.
- Código só entra no sandbox após ação do utilizador.

17.7 Testes multimodais

- Dataset próprio com pseudocódigo manuscrito/digital, código, diagramas e ruído.
- Variações de rotação, iluminação, compressão e resolução.
- Golden tests para estrutura e indentação.
- Testes adversariais de imagens malformadas e grandes.

- Métricas de character/word error rate e acerto estrutural.
- Testes desktop, Android e iOS.

Gate MULTI-1

- Em imagens digitais nítidas, OCR $\geq 95\%$ de caracteres no dataset de referência.
- Baixa confiança é mostrada e requer confirmação.
- Conversão para IDE preserva o original e cria uma nova versão.
- Nenhuma imagem ou texto sai do dispositivo.
- A explicação liga conceitos, curso e dicionário.

18. Fase 12 — Games e pausas

Games permanece uma aba separada. Esta fase só começa após os gates pedagógicos anteriores.

- Sudoku e Minesweeper com guardar/retomar.
- Três dificuldades calibradas.
- Teclado, toque e acessibilidade.
- Modo diário gerado localmente.
- Temporizador de pausa opcional.
- Lembrete discreto para regressar ao plano.
- Estatísticas locais.
- Sem animações intrusivas, anúncios ou recompensas manipuladoras.
- Games não concede mastery nem substitui exercícios.

Gate GAMES-1

- Jogos não degradam arranque, bateria ou estabilidade do ensino.
- Estado é isolado do progresso pedagógico.
- Todas as grelhas geradas são válidas e solucionáveis.

19. Fase 13 — Hardening e release 1.0

Segurança

- Threat model atualizado.
- Fuzzing de parsers, packs, imagens e import/export.
- Testes de sandbox escape.
- Rotação e backup de chaves.
- Assinatura dos packs e, quando disponível, dos binários.

- SBOM e análise de dependências.

Desempenho

Orçamentos no hardware de referência:

- arranque desktop P95 < 3 s após primeira inicialização;
- arranque mobile P95 < 5 s;
- navegação comum < 100 ms;
- pesquisa local P95 < 500 ms;
- autosave sem perda e sem stutter;
- indexação e OCR canceláveis e em background;
- consumo de memória documentado por pack/modelo.

Qualidade

- regressão completa unitária, integração e E2E;
- testes de migração desde versões suportadas;
- testes visuais de temas, DPI e ecrãs;
- testes de acessibilidade;
- teste offline físico;
- soak test prolongado;
- validação editorial amostrada;
- instalador, desinstalador, backup e rollback.

Release

- Canal stable e canal preview opcionais.
- Release notes orientadas ao utilizador.
- Guia de recuperação.
- Manifestos com hashes.
- EXE e APK finais; IPA apenas após build e assinatura num Mac.
- Conteúdo inicial empacotado e packs opcionais.

Gate RELEASE-1

- Zero falhas críticas abertas.
- Zero perda de dados nas migrações testadas.
- Zero bypass conhecido do sandbox.
- Todas as jornadas críticas funcionam offline.
- Pesquisa, recomendação e OCR cumprem benchmarks.
- Dispositivos físicos desktop/Android/iOS validados conforme suporte declarado.

20. Matriz de testes

Camada	Testes
Domínio	Regras, invariantes, propriedades e determinismo
Aplicação	Casos de uso, cancelamento, erro e idempotência
Base de dados	Migrações, transações, índices, corrupção e rollback
Pesquisa	Golden queries, relevância, diversidade e quarentena
Aprendizagem	Simulações, calibração, pré-requisitos e explicabilidade
Conteúdo	Schema, licença, proveniência, duplicação e exatidão
IDE	Diagnósticos, execução, debugger, timeout e escape
OCR	Qualidade, layout, confiança, formatos e adversarial
UI	Visual regression, teclado, toque, DPI e acessibilidade
Mobile	APIs, lifecycle, armazenamento, rotação e bateria
Release	Hashes, assinatura, instalação, update e desinstalação

21. Indicadores de produto locais

Sem telemetria cloud, o utilizador pode consultar e exportar métricas locais:

- retenção por conceito;
- taxa de acerto sem dicas;
- tempo até primeira solução correta;
- transferência entre contextos;
- evolução semanal;
- cumprimento do plano;
- cobertura do currículo;
- revisões vencidas;
- projetos concluídos;
- distribuição dos tipos de erro.

Para avaliar o produto durante desenvolvimento usam-se datasets sintéticos e voluntários, nunca recolha silenciosa.

22. Riscos e mitigação

Risco	Mitigação
Excesso de funcionalidades	Progressive disclosure e gates por jornada
Conteúdo amplo mas fraco	Quarentena, revisão e métricas de qualidade
Modelos pesados	Modelo-base pequeno e packs opcionais
Pesquisa personalizada cria bolha	Misturar relevância global e exploração
Progresso dá falsa precisão	Intervalos de confiança e evidência visível
Scraping quebra ou viola licença	Adapters allowlisted, metadata e cache
Update compromete offline	Broker separado, assinatura e rollback
OCR altera código	Preview, confiança e confirmação obrigatória
Debugger abre superfície de ataque	Processo isolado e protocolo restrito
Divergência desktop/mobile	Contratos partilhados e testes de paridade
iOS fica por validar	Gate obrigatório em Mac e dispositivo real
Base cresce demasiado	Packs temáticos, índices incrementais e quotas

23. Planeamento e estimativa

Este roadmap representa trabalho de produto “AAA”, incluindo conteúdo, testes e polimento. Não deve ser prometido como uma alteração de poucos dias.

- **Uma equipa pequena e execução maioritariamente sequencial:** aproximadamente 12–18 meses, dependendo do volume editorial.
- **Três fluxos paralelos com integração disciplinada:** aproximadamente 8–12 meses.
- **Conteúdo avançado completo:** continua como programa editorial após o release 1.0.

As estimativas devem ser recalculadas após as fases 0, 3, 6 e 11. O conteúdo é o maior fator variável.

Marcos de release sugeridos

Release	Conteúdo
0.19	Fundação, design system e navegação

Release	Conteúdo
0.20	Mastery Engine e planeador
0.21	Academia e cursos-base
0.22	Pesquisa/BD v2
0.23	Packs e atualizador semanal
0.24	IDE e debugger
0.25	Dicionário, cards e bibliografia v2
0.26	Tutor offline
0.27	Projetos e portefólio
0.28	Mobile/iOS e transferência
0.29	Caixa multimodal de texto/imagem
0.30	Games, hardening e release candidate
1.0	Release estável após todos os gates

24. Próximo passo imediato

A execução deve começar pela **Fase 0**, criando o baseline reproduzível e os contratos de métricas. Em seguida inicia-se a **Fase 1**, sem alterar ainda algoritmos pedagógicos. Isso permite renovar a interface sem misturar defeitos visuais com alterações de recomendação.

O primeiro incremento demonstrável deve entregar:

1. nova navegação adaptativa;
2. design system claro/escuro/alto contraste;
3. continuação da última atividade;
4. command palette;
5. persistência de layout e scroll;
6. testes DPI, teclado e mobile;
7. métricas de baseline para comparar o que melhorou.

Depois desse gate, o botão **Melhor próximo passo** torna-se o centro do produto e conduz as fases seguintes.

25. Decisão final de estratégia

A estratégia recomendada é construir primeiro o sistema que decide e apresenta bem, depois aumentar aquilo que ele consegue ensinar. O Appendix só deve receber pesquisa mais sofisticada, grandes volumes de conteúdo, tutor e multimodal quando possuir contratos estáveis, qualidade mensurável e uma interface que não esconda o valor dessas capacidades.

A caixa de texto/imagem deve ser integrada como ferramenta pedagógica, não como chat genérico. Ela recebe um trecho, confirma o OCR, compreende estrutura, consulta conhecimento aprovado, ensina o conceito e transfere um rascunho controlado para o IDE. Essa integração é o que a torna útil, privada e coerente com o restante produto.

O resultado pretendido é um ciclo único:

Objetivo → diagnóstico → plano → conteúdo → prática → debug → avaliação → revisão → projeto → domínio demonstrado.